JKSCI

Journal of The Korea Society of Computer and Information

Design and PoC Validation of a Secure On-Premises Meeting Intelligence Pipeline for Construction Enterprises

Jaehwan Kim[1] · Jaeseong Yun[2] · Jongseok Lee[3] · Kyungmin Kim[4]

[1]aSSIST, Seoul, Republic of Korea; [2]XAIKOREA Inc., Seongnam, Republic of Korea; [3]RAD CNS Co., Ltd., Republic of Korea; [4]Security Platform, Republic of Korea

First and Corresponding Author: Jaehwan Kim (kjh861213@naver.com)

[Abstract]

This study evaluates transcript-artifact recovery within an on-premises meeting intelligence proof of concept for construction enterprises. The evaluation concerns independently decoded audio chunks, rather than whole-system security or Korean-domain recognition accuracy. With the ASR model and decoding settings fixed, we compared full restart, syntax-based reuse, and combined output-integrity and execution-condition validation. The first 120 seconds of 16 AMI test recordings were divided into 30-second chunks. All 912 executions terminated: 48 no-fault controls and 864 fault-injection runs. Full restart and combined validation preserved the uninterrupted reference transcript in every fault-injection run; syntax-based reuse committed incorrect transcripts in 96 text-corruption runs. After stop-only interruption following the third saved chunk, the mean of recording-level median recovery latencies was 26.19 seconds for both reuse policies and 83.52 seconds for full restart. The paired reduction for combined validation was approximately 57.3 seconds. The contribution is a controlled characterization of the relationship between transcript preservation, validation scope, and recomputation cost, supported by failure-inclusive execution records. Output-hash validation alone can detect the corruption used in this experiment. The incremental benefit of execution-condition checks therefore remains a separate question for the ongoing output-hash-only comparison.

▸ Keywords: On-Premises AI, Meeting Intelligence, Artifact Integrity, Checkpoint Recovery, Fault Injection

[요약]

본 연구는 건설기업용 온프레미스 회의 지능화 개념검증 시스템에서 전사 산출물의 장애 복구 동작을 평가한다. 평가 범위는 독립적으로 추론하는 음성 청크의 산출물 복구이며, 전체 시스템 보안이나 한국어 건설회의 인식 정확도는 포함하지 않는다. 동일한 음성인식 모델과 추론 설정에서 전체 재시작, 구문 기반 재사용, 출력 무결성과 실행조건을 함께 검사하는 재사용을 비교하였다. AMI 테스트 녹음 16개의 처음 120초를 30초 청크로 나누어 무장애 대조 48회와 장애 주입 864회, 총 912회를 실행하였다. 모든 실행이 종료되었으나 구문 기반 재사용의 텍스트 변조 조건 96회는 기준 전사와 다른 결과를 확정하였다. 전체 재시작과 결합 검증은 모든 장애 조건에서 기준 전사를 보존하였다. 세 번째 청크 저장 후 단순 중단 조건에서 녹음별 복구 지연시간 중앙값의 평균은 두 재사용 정책 모두 26.19초, 전체 재시작은 83.52초였다. 본 연구의 기여는 실행 종료와 전사 보존을 구분하고, 검증 범위와 유효 청크의 잔존량에 따른 재추론 비용을 실패를 포함한 기록으로 분석한 데 있다. E1의 텍스트 변조는 출력 해시만으로도 검출되므로 실행조건 검사의 추가 효과는 별도의 진행 중인 비교로 평가해야 한다.

▸ 주제어: 온프레미스 인공지능, 회의 지능화, 산출물 무결성, 체크포인트 복구, 장애 주입

I Introduction

When a transcription job is interrupted, recovery should reproduce the uninterrupted output while avoiding unnecessary recomputation. Previously generated chunk transcripts can serve as checkpoints. However, accepting partial files, readable files with altered content, or outputs from incompatible tasks may let a job finish with an incorrect transcript. We therefore evaluate execution completion, transcript preservation, and recovery cost separately.

Whisper and WhisperX provide foundations for speech recognition and temporal alignment [1,2]. Recovery policies determine which artifacts produced before an interruption can be trusted and which chunks require recomputation. We investigate whether recovery can preserve the transcript of an uninterrupted reference run using the same model while reducing recomputation after specified faults. Because this process may also preserve recognition errors in the original output, we distinguish transcript preservation during recovery from recognition accuracy against a human reference transcript.

The title retains the application context of our earlier construction-meeting proof of concept (PoC) [17]. That PoC integrated speech recognition, speaker diarization, summarization, storage, and key management. This paper evaluates one subsystem: deciding which saved transcript chunks can be reused after interruption. The equipment and interface in Appendices D and F document the application setting; the quantitative evidence concerns public English AMI excerpts. Encryption effectiveness, resistance to host compromise, end-to-end meeting understanding, and recognition accuracy for Korean construction meetings are outside the demonstrated scope.

We address two research questions (RQs). RQ1 examines how transcript preservation varies across reuse policies under specified file-corruption conditions. RQ2 examines how recomputation and latency vary with fault type and interruption position among runs that preserve the transcript. Both questions are investigated using the completed E1 experiment. The incremental benefit of adding execution-condition checks to hash checks, and comparisons with existing workflows, require separate validation.

The contribution is an empirical characterization of transcript-preserving recovery for independently decoded audio chunks. First, the study separates a process finishing normally from the recovered transcript matching its uninterrupted reference, revealing 96 incorrect commits that an exit-status-only evaluation would miss. Second, it relates recomputed audio and paired recovery latency to the amount of valid output retained at each interruption position. Third, it provides a frozen fault matrix, failure-inclusive records, and reproducible descriptive aggregates. These contributions concern experimental characterization and traceability; hashing, checkpointing, and task-dependent caching are established principles.

The central result is conditional: content validation prevents reuse of the altered outputs tested here, and recovery saves computation only when valid chunks remain. E1 does not isolate execution-condition check P from output-hash check H. The evidence map in Appendix G identifies which findings support each claim and which comparisons remain pending.

II Related Work and Study Scope

1 Speech Processing and Recovery Preservation

Whisper is an automatic speech recognition (ASR) model trained on large-scale weakly supervised audio-transcript data [1]. WhisperX adds alignment to locate words in time [2], while pyannote.audio supports speaker diarization by grouping speech

segments by speaker [3]. These tasks address what was said, when it was said, and who said it. Our focus is whether a generated transcript remains unchanged after interruption and restart. We therefore hold the recognition model and chunk boundaries constant and vary only the recovery policy.

2 Checkpoint Recovery and Fault Injection

Checkpointing allows computation to resume from saved intermediate state rather than restart from the beginning. In their survey of checkpoint- and log-based recovery, Elnozahy et al. identify recovery-state consistency and the amount of repeated computation as central concerns [4]. In our implementation, each completed audio-chunk transcript serves as an application-level checkpoint. If three of four chunks remain valid, only the final chunk requires processing. A retained output must nevertheless be recomputed if it is corrupt or was generated under conditions incompatible with the current input. Suitability for reuse therefore matters as much as the presence of stored output.

Fault injection deliberately introduces interruptions or corruption to assess detection and recovery behavior. Hsueh et al. describe an experimental approach that coordinates injection targets, mechanisms, and observation methods [5]. We apply three fault types to the same audio rather than wait for naturally occurring failures. Stop-only interruption tests reuse of valid results; JSON truncation tests the handling of unreadable files; and content modification tests validation of readable but altered content. Holding other conditions constant allows policy outcomes to be related to the checks each policy performs. Equal run counts across conditions support controlled comparison but do not represent operational fault frequencies.

3 Execution-Condition-Based Caching and Crash Consistency

Caching retains computed results for later reuse. Its central requirement is that a stored result correspond to the task currently requested. Bazel's remote cache distinguishes result metadata associated with inputs and an execution specification from the storage of output content [6]. An unchanged transcript file, for example, is not necessarily suitable for reuse if the input audio has changed. We apply this established distinction to transcript chunks by checking both content integrity and compatibility with the current execution conditions.

Nextflow, introduced by Di Tommaso et al., supports computational reproducibility by managing execution environments and workflows [7]. Cache reuse depends on task hashes incorporating inputs, scripts, containers, and related information, together with stored outputs [8]. DVC similarly manages stage re-execution and output restoration using declared dependencies and parameters [9]. The experimental data-lineage feature introduced in Nextflow 25.04 records relationships between output checksums and the tasks that produced them [10]. Snakemake's between-workflow cache also identifies results using inputs, parameters, and the software environment [11]. Provenance recording, reuse decisions, and output-content validation serve different roles depending on implementation and configuration; feature descriptions and actual fault responses must therefore be compared separately.

Crash-consistency research examines how requested writes relate to the state recoverable after a crash. Pillai et al. show that application update procedures depend on filesystem persistence properties and that these dependencies affect crash consistency [12]. An interruption between updates to a transcript file and its validation record can therefore leave the two in inconsistent states. E1 injects interruption after acknowledgment of a completed chunk save, whereas a separate file experiment terminates the writing process between individual storage steps. The two experiments address reuse decisions and storage ordering at distinct levels.

4 Scope of On-Premises Deployment

On-premises deployment allows an organization to manage models and data within its own environment. NIST's zero-trust guidance emphasizes verifying trust in subjects and resources independently of physical or network location [13]. Our recovery evaluation assumes trusted inputs, models, execution code, and controller. Authentication, encryption, key separation, administrative privileges, and log management remain separate security requirements for an operational system.

Existing checkpoint and workflow systems already address recovery and task-dependent reuse. Our distinction is the evaluation question: whether a normally completed recovery preserves the exact uninterrupted transcript, and how this outcome changes with the fault and the remaining valid chunks under the same ASR worker. Table 1 separates established mechanisms, observations made here, and comparisons not yet performed. It is not a benchmark ranking of workflow tools.

Table 1. Established mechanisms and the scope of the present evidence

Study or framework	Main focus	Evidence and comparison boundary
Whisper and WhisperX [1,2]	Recognition and alignment	Whisper is E1's ASR foundation
pyannote.audio [3]	Speaker diarization	Diarization component of the earlier application
Bazel remote cache [6]	Task- and content-based reuse	Established task/content distinction; no superiority comparison performed
Nextflow · DVC · Snakemake [8,9,10,11]	Reuse based on inputs, settings, and environment	Task-aware reuse is established; equivalent-configuration fault benchmark pending
Application crash consistency [12]	Storage updates and persistence	E3a observes file states after process termination, not power-loss durability
This study	Failure-inclusive ASR recovery characterization	Same-worker fault matrix; exact transcript preservation and paired cost; P benefit not isolated in E1

III Selective Recovery with Execution-Condition Validation

1 Tasks and Trust Boundaries

An artifact is a file containing the transcript and associated metadata generated for an audio chunk. Each recording is divided into four chunks. Artifact A_i for chunk i contains a transcript string (text), a list of segment timing and related information (segments), and the execution conditions under which it was generated (binding). Artifacts are stored in JavaScript Object Notation (JSON). A separate validation record L_i stores the file's SHA-256 (Secure Hash Algorithm 256-bit) hash and execution conditions. This hash detects changes to the file bytes; it does not authenticate the author of the validation record as a digital signature would.

Execution metadata, called binding in the stored JSON, has exactly four keys: id, audio_sha256, model_sha256, and protocol_sha256. The expected binding is constructed from the frozen chunk manifest and protocol, rather than inferred from the stored output. Combined validation requires both the artifact binding and the validation-record binding to equal that expected dictionary. The protocol hash covers the inference settings. Code and container-image hashes are frozen separately; they are not additional fields in E1's binding. Appendix G gives the decision procedure and a concrete schema example.

We assume that the controller, input specification, model, execution code, and validation records remain unchanged from their specified states. E1 alters output files while preserving their validation records to test whether the recorded validation information detects those changes.

Fig. 1 illustrates how audio chunks and execution conditions determine the artifacts available for reuse during recovery. Its three-dimensional layout distinguishes the logical components.

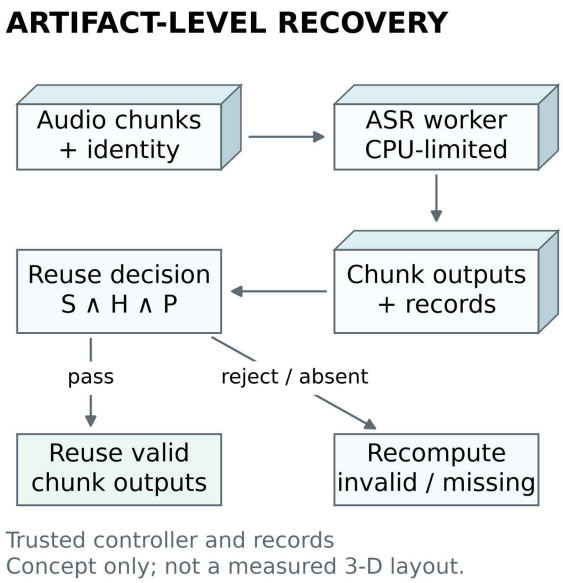


Fig. 1. Conceptual three-dimensional view of transcript-artifact generation, validation, and reuse.

2 Reuse Decisions

Reuse validation addresses three questions. First, can the file be parsed and its required fields interpreted? The syntax condition S(A_i) is a minimal format check: it requires parseable JSON, a string-valued text field, and a list-valued segments field. An empty transcript is permitted for a silent chunk. Second, have the file bytes changed since storage? The integrity condition H(A_i,L_i) compares their current hash with the value in the validation record. Third, was the artifact generated for the task now being performed? The execution-condition check P(A_i,L_i,B_i∗) compares the conditions recorded in the artifact and validation record with those required by the current task.

For example, a current task may require model B while a transcript from model A remains beside its matching validation record. The JSON and output hash can both be valid, yet the model_sha256 field differs from the expected value. This is an execution-condition mismatch, distinct from E1's altered-text fault. A hash over output bytes alone cannot identify that mismatch when the accompanying record belongs to the same old output.

The combined reuse rule is $R_i = S(A_i) \wedge H(A_i,L_i) \wedge P(A_i,L_i,B_i*)$. Here A_i is the artifact, L_i its validation record, and B_i∗ the expected execution metadata. S checks the JSON object and the types of text and segments; H compares SHA-256 of the stored artifact bytes with L_i.output_sha256; P requires A_i.binding = L_i.binding = B_i∗. All conditions must hold. This is an eligibility rule, not a test of whether the transcript matches human speech.

For each of the four chunks, the worker constructs the expected binding, tries the checks required by the selected policy, and either reuses the artifact or runs inference. Missing files or records, unreadable JSON, missing required keys, and incompatible field types make the artifact

ineligible under the relevant checks. The worker writes a new artifact and then its validation record for every recomputed chunk. After all chunks are available, the evaluator records process status and exact transcript equality separately. Appendix G follows the archived source rather than adding checks retrospectively.

Full restart reruns inference for every chunk, whereas the syntax-based policy reuses outputs satisfying S. E1's combined-validation policy applies S, H, and P. The output-hash-only policy in the supplementary experiment applies S and H and is designed to assess the incremental effect of P through comparison with combined validation.

Fig. 2 maps the questions asked by the three checks to the policies compared experimentally. File readability, preservation of stored content, and suitability for the current execution are distinct properties.

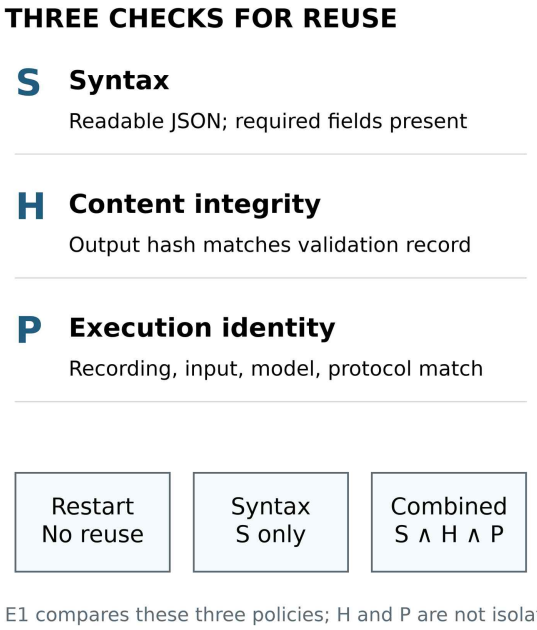


Fig. 2. Roles of syntax (S), content-hash (H), and execution-condition (P) checks and the policies compared in E1.

Syntax-based reuse is a deliberately minimal comparator, not a representative implementation of every existing checkpoint system. Full restart supplies the recomputation baseline. H alone is sufficient for E1's altered-text condition, so a difference between syntax-based reuse and combined validation is not evidence of P's incremental benefit.

Here, a result is committed when a recovery job writes its result files and terminates. If the committed result differs from the reference transcript, the run is classified as a correctness failure and recorded as incorrect_commit in the raw data.

3 Storage and Result Commit

Interpreting the state left by an interrupted write requires distinguishing the transcript-file update from the validation-record update. The E1 worker writes the transcript to a temporary file and replaces the output file, then updates the validation record in the same manner. Replacing each file separately does not commit the pair as a single operation. An interruption between these updates can leave the artifact and its record in inconsistent states. E1 terminates the worker after acknowledgment of completed chunk processing; the separate file experiment examines interruptions at the intervening storage boundaries.

After restart, the worker validates each chunk, reuses eligible outputs, and runs inference for the remainder. Once all chunks are available, it records the termination status and compares the list of chunk-level transcript strings with the uninterrupted reference. Exact equality of these lists is E1's correctness criterion. The supplementary experiment additionally distinguishes transcript-string equality from equality of the complete structured output.

4 Interpretation of Decisions and Dependency Scope

The evaluation distinguishes reuse eligibility, preservation of the reference transcript, and recognition quality against a human reference. Reuse eligibility concerns whether an artifact is suitable for the current task; preservation concerns whether the recovered output equals the uninterrupted output; and recognition quality

concerns whether that output accurately represents the speech. Reproducing a sentence originally misrecognized by the model preserves the transcript but also preserves its recognition error. Timing metadata may likewise differ even when the text is identical. E1 therefore assesses exact equality of chunk-level strings, while the supplementary experiment separately records structured-output equality and execution-condition compliance.

The current implementation decodes each 30-second chunk independently, without using transcripts from other chunks as input. Reprocessing one chunk therefore does not require recomputation of its neighbors. In long-form transcription conditioned on preceding text, however, changing an earlier chunk may affect subsequent decoding. Such a system must record both the context and the generation conditions and determine whether recomputation needs to propagate to later chunks. Our chunk-level recovery findings apply to the independent-processing setting.

5 Operational Controls

Experiments run one job at a time in an isolated container with read-only access to the model, code, and inputs. Central processing unit (CPU) usage is capped at 2 CPUs, and both the memory limit and the combined memory-plus-swap limit are set to 8 GiB. Graphics processing unit (GPU) access and external networking are disabled. The controller monitors existing service states and available memory. If a previously healthy service deteriorates or resource criteria are violated, it stops only the experimental job. This monitoring assesses service execution state.

IV Experimental Design and Analysis

1 Data and Fixed Conditions

The Augmented Multi-party Interaction (AMI) corpus contains public multi-party meeting recordings [14]. We selected 16 test recordings from a split manifest pinned to a fixed revision [15]. For each recording, we retained the first 120.000 seconds, including silence, and divided them into four consecutive 30-second chunks. Inputs were 16 kHz, single-channel, 16-bit pulse-code modulation (PCM) audio. The original waveforms and fixed intervals were preserved without alteration. The total duration of unique input audio was 1,920 seconds.

The recordings are EN2002a-d, ES2004a-d, IS1009a-d, and TS3003a-d: 16 recordings in four series, each contributing the interval [0,120) seconds. Their four chunk intervals are [0,30), [30,60), [60,90), and [90,120) seconds. Thus, the 912 executions reuse only 32 minutes of unique audio. Repetitions characterize technical timing variation and do not increase the number of independent recordings or series [16]. Test-data results had been inspected before the follow-up studies were designed.

Inference used Whisper large-v3-turbo on a CPU with two threads. The language was fixed to English, temperature to 0, and beam size to 1; fp16 and conditioning on preceding chunk text were disabled. SHA-256 identifiers for the model weights, execution image, protocol, input manifest, and schedule were frozen. These settings and identifiers specify the conditions required to reproduce the experiment.

The E1 worker calls load_model and transcribe directly in the Python implementation of OpenAI Whisper. It implements only the ASR stage and uses AMI Mix-Headset as the fixed input channel. The shared aarch64 host has 10 Cortex-X925 cores and 10 Cortex-A725 cores, but the experimental container was limited to the equivalent of 2 CPUs. The frozen container image identifies the execution environment; hashes of the model, execution code, and input manifest are retained alongside it. Appendix C describes these identifiers.

2 Fault Matrix and Execution Order

The main experiment, E1, crosses three policies, three fault types, two interruption positions, and

three technical replicates. It contains 16×3×3×2×3=864 fault-injection runs and 16×3=48 no-fault controls, yielding 912 runs in total. For each recording, an uninterrupted run over all chunks supplies the reference transcript, against which the no-fault controls were checked. The remaining runs followed a frozen schedule generated with seed 20261026.

After the worker saves the first or third chunk and sends a readiness acknowledgment, the controller terminates it with SIGKILL. In the stop-only condition, recovery begins without further changes to the files. JSON truncation removes half of the first chunk's JSON bytes. Content modification replaces that chunk's text with CORRUPTED OUTPUT while retaining valid JSON structure. Both corruption conditions leave the validation record unchanged. The intended SIGKILL is part of the fault-injection protocol and is distinguished from unexpected execution failures, including abnormal worker termination or monitoring failure.

Each recording first has one uninterrupted full-restart run, whose ordered list of four transcript strings serves as its reference. The other two no-fault policy runs must match that list. This yields 16 reference runs within the 48 no-fault controls, not 48 independent reference recordings. The fault runs then launch the same worker, wait for the first or third chunk-save acknowledgment, terminate it with SIGKILL, apply the scheduled file fault if any, and restart. Each final transcript is compared with the corresponding recording's reference. Reference IDs and all 912 scheduled IDs are supplied in the companion material.

Table 2. Design of the main E1 experiment

Item	Specification
Input	First 120 seconds of each of 16 recordings; 4 series
Processing unit	Four 30-second chunks; no cross-chunk transcript conditioning
Policies	Full restart; syntax checks; combined validation (S+H+P)
Fault	Stop only; JSON truncation; text modification preserving syntax
Interruption position	After saving chunk 1 or 3
Replicates and runs	3 technical replicates per fault condition; 864 fault runs; 48 no-fault controls
Correctness	Exact chunk-string equality with the same recording's clean full-restart transcript
Limits	600-second limit per job; unexpected failures and partial results retained

Fig. 3 summarizes E1's fault-injection sequence. Post-interruption modification targets the first chunk's artifact, and the recovered transcript is compared with the uninterrupted reference.

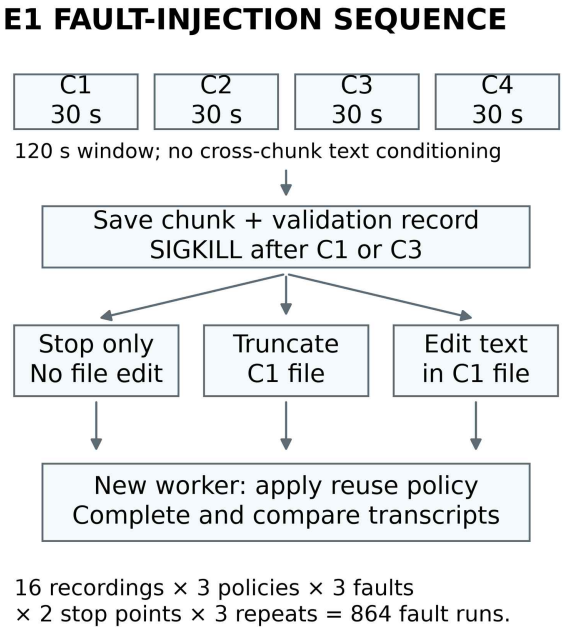


Fig. 3. E1 sequence of storage, interruption, modification, and recovery. C1–C4 are 30-second audio chunks; interruption occurs at C1 or C3 after the save acknowledgment.

The no-fault controls comprise 48 runs: one for each combination of 16 recordings and 3 policies. Fault injection comprises 16 recordings × 3 faults × 2 interruption positions × 3 policies × 3 replicates = 864 runs. The denominator for fault-recovery outcomes is 864.

3 Outcome Measures and Conditional Timing Analysis

Recovery correctness is assessed against the transcript produced from the same input and model without a fault. Comparing the recovered list of chunk-level strings with this reference identifies changes introduced during recovery. A run that terminates normally but returns a different list is classified as an incorrect commit. Execution failures, timeouts, and no-fault controls are recorded separately.

Recomputed audio duration C is the sum of the durations of chunks inferred after the fault. Post-fault latency L runs from immediately before SIGKILL to immediately after the recovery worker terminates and both its result and the reference transcript have been read. It includes corruption, worker restart, model loading, validation, inference, and storage. Total elapsed time T starts immediately before the first worker launch and ends at the same point, thereby including pre-interruption inference. Writing the final result.json, preparing the container, and waiting between batches are separate operational costs. Because L includes experimental control and result reading, relating it to user-perceived recovery time requires consistent measurement boundaries. L is the primary timing measure; Table 5 and Appendix E provide a post hoc analysis of the recorded T values.

To account for differences in speech content and processing difficulty, we compare policies within the same recording. For policy a relative to full restart, the differences are ΔC=C_a-C_restart and Δ L=L_a-L_restart; negative values indicate lower cost. We first calculate paired differences for the same recording, fault, interruption position, and replicate, then take the median of the three differences for each recording. We finally average these recording-level medians with equal weights. Taking the median limits the influence of a single large timing fluctuation. A recording is eligible only if both policies preserve its reference transcript in all three replicates; every condition in Table 4 includes 16 recordings. This order of aggregation can produce a different estimate from the difference between separately calculated policy medians.

Recovery performance encompasses both correctness and speed. Averaging latencies from runs that quickly return incorrect transcripts can give a misleading picture of performance. We therefore retain all correctness failures in Table 3 and condition timing comparisons on correct recovery, explicitly identifying excluded conditions and denominators. Reusing k chunks of 30 seconds saves 30k seconds of recomputed audio. Elapsed time also depends on model loading, validation, storage, and host state. Reporting audio duration and latency separately reveals how reduced computation translates into time savings.

4 Role of Supplementary Validation

The supplementary studies were designed after intermediate E1 results were inspected. They address questions that E1 cannot resolve, especially reuse of an intact artifact and matching record from incompatible input, model, or decoding conditions. Their protocols and denominators remain separate from E1.

E2 denotes the separately frozen supplementary main study: 960 main executions (64 no-fault controls and 896 fault cases), with 32 donor-generation executions reported separately. Its output-hash-only and combined-validation policies are intended to isolate the effect of P. E2 was incomplete at this manuscript's evidence cutoff and contributes no main-study results here. E3a denotes the synthetic storage-boundary study (14 process terminations and 2 uninterrupted controls). Two separate ASR development jobs establish

integration feasibility. Appendix G maps these evidence types and their limitations.

V Results

1 Execution Completion and Transcript Preservation

All 912 runs terminated normally, with no unexpected execution failures or timeouts. The 816 runs matching the reference transcript comprise 48 no-fault controls and 768 fault-injection runs. The remaining 96 of the 864 fault-injection runs committed incorrect transcripts.

Each fault-policy cell in Table 3 contains 96 runs: 16 recordings, two interruption positions, and three replicates. Under content modification, syntax-based reuse committed an incorrect transcript in all 96 runs. The injected change altered only text while preserving JSON syntax and field types, so the policy accepted the modified artifact. These correctness failures occurred despite normal execution termination. Under stop-only interruption and truncation, all three policies preserved the transcript in all 96 runs per cell. Combined validation and full restart also preserved it in all 96 content-modification runs.

Table 3. E1 outcomes by fault type and policy

Fault	Policy	Runs	Transcript matches	Incorrect commits
Stop only	Full restart	96	96	0
Stop only	Syntax checks	96	96	0
Stop only	Combined	96	96	0
Truncated JSON	Full restart	96	96	0
Truncated JSON	Syntax checks	96	96	0
Truncated JSON	Combined	96	96	0
Text edit	Full restart	96	96	0
Text edit	Syntax checks	96	0	96
Text edit	Combined	96	96	0
Total	Fault runs only	864	768	96

The 96 incorrect commits are repeated observations of the same corruption-policy combination. The key finding is that identical normal-termination status did not imply identical transcript preservation.

Fig. 4 illustrates the distinction between execution completion and transcript preservation. All 96 syntax-based runs under content modification terminated, but none preserved the reference transcript.

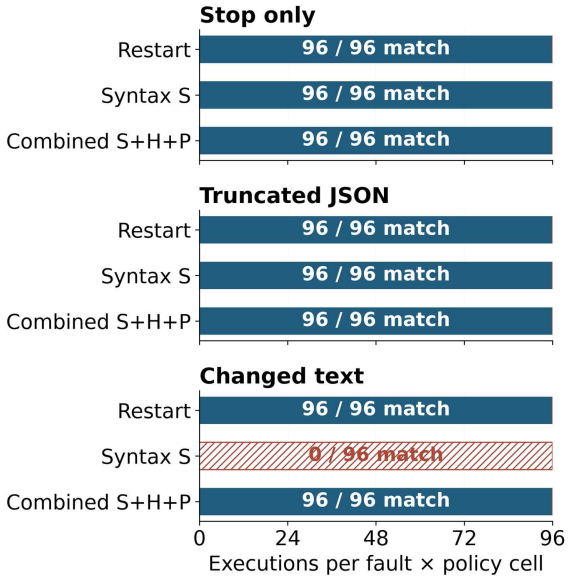


Fig. 4. Transcript preservation by policy in E1's 864 fault-injection runs. Each bar represents 16 recordings × 2 interruption positions × 3 replicates = 96 runs. Hatching denotes incorrect transcript commits; the 48 no-fault controls are excluded. The counts include technical replicates of the same recordings.

2 Recomputation and Latency in Correct Recovery

Table 4 compares combined validation with full restart. In every row, both policies preserved the transcript in all three paired replicates for each of the 16 recordings. After stop-only interruption following the first saved chunk, combined validation processed the remaining 90 seconds, saving 30 seconds of audio recomputation. After the third saved chunk, it processed only the final 30 seconds, saving 90 seconds. The corresponding latency differences were -16.061 and -57.312 seconds.

When the first chunk was truncated or modified at the first interruption position, it also required recomputation. All four chunks, totaling 120 seconds, were therefore processed again. Recomputation savings were zero, and the respective latency differences of -0.001 and +0.023 seconds were close to zero. When the first chunk was corrupted after three chunks had been saved, the second and third chunks remained reusable, saving 60 seconds of audio recomputation.

Table 4. Conditional differences between combined validation and full restart. n: eligible recordings.

Fault	Saved chunks	Recomp. (s)	ΔC (s)	ΔL (s)	n
Stop only	1	90	-30	-16.061	16
Stop only	3	30	-90	-57.312	16
Truncated JSON	1	120	0	-0.001	16
Truncated JSON	3	60	-60	-41.366	16
Text edit	1	120	0	+0.023	16
Text edit	3	60	-60	-41.439	16

ΔL is the equally weighted mean of the recording-level medians of three paired differences. Full restart recomputes 120 seconds after the fault in every E1 condition. Syntax-based reuse achieved similar recomputation savings under stop-only interruption and truncation. Under content modification, however, it produced correctness failures for all 16 recordings at both positions, leaving no eligible observations for a timing comparison conditional on correct recovery.

Fig. 5 adds syntax-based reuse to the comparison, using the aggregation rules applied to combined validation in Table 4. Both reuse policies yield similar savings when recovery is correct. The value of combined validation therefore depends on the conditions under which it permits reuse, as well as the magnitude of the resulting savings.

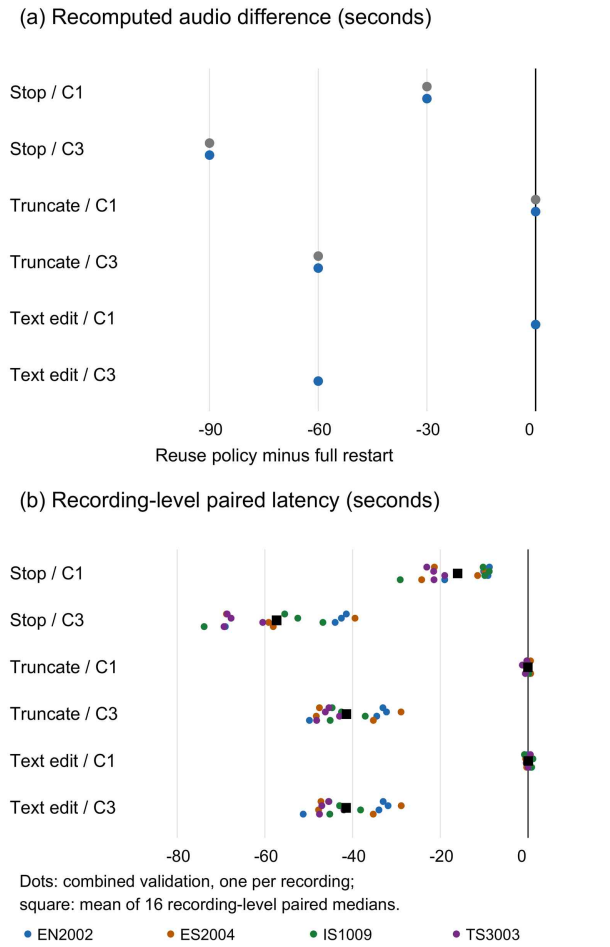


Fig. 5. Differences relative to full restart in (a) recomputed audio duration and (b) recording-level paired latency. In (a), gray denotes syntax-based reuse and blue denotes combined validation; correctness-failure conditions are excluded. In (b), points show recording-level medians of three paired differences for combined validation, and squares show the equally weighted mean across 16 recordings. Colors distinguish the four meeting series.

Table 5 presents a post hoc analysis of the archived E1 records using total elapsed time T, which includes pre-interruption cost. For stop-only interruption at position 3, post-fault latency L was 83.52 seconds for full restart and 26.19 seconds for each reuse policy. The corresponding T values were 145.20, 87.80, and 87.93 seconds. The savings common to both reuse policies reflect reuse of valid artifacts. Additional validation must therefore be assessed in terms of both the conditions under which it preserves correctness and the cost it introduces.

For combined validation at position 1, the paired total-time differences ΔT are +0.066 seconds for JSON truncation and +0.009 seconds for content modification. They describe variation near zero in these observations. They are not obtained by subtracting the rounded absolute-time columns in Table 5: paired differences are formed before taking replicate medians.

Table 5. Post hoc analysis of total elapsed time T for correct recovery

Fault	Pos.	Full restart	Syntax checks	Combined	Runs per policy
Stop only	1	104.01	87.73	87.97	48
Stop only	3	145.20	87.80	87.93	48
Truncated JSON	1	103.82	103.90	103.80	48
Truncated JSON	3	145.36	103.68	103.95	48
Text edit	1	103.77	N/A	103.81	48
Text edit	3	145.24	N/A	103.76	48

Estimator: let T_a,m,r be total time for policy a, recording m, and matched replicate r within one fault-position condition. Each absolute-time column is mean_m median_r(T_a,m,r). The paired contrast is ΔT_a = mean_m median_r(T_a,m,r − T_restart,m,r). These estimators need not equal a difference between column means. Each eligible condition has 16 recordings with 3 correct replicates each. N/A denotes the 96 incorrect syntax-based runs across the two altered-text conditions; their 8417.51 seconds remain in the failure-inclusive cost total. Table 4 uses the analogous paired estimator for post-fault latency L.

3 Storage-Boundary and Development Integration Observations

All 16 E3a cases produced the expected storage states and policy decisions. In three cases, a stale but valid artifact remained alongside its matching hash-validation record. Syntax-based and output-hash-only policies each accepted all three for incorrect reuse, whereas full restart and combined validation permitted none within this case set. An output-hash match alone can therefore accept an artifact generated for a different task. Appendix C lists the states and identifiers for each storage-boundary case. E3a uses synthetic file contents and execution-condition labels, but terminates the actual file-writing process. These observations are distinct from the two development integration jobs that interrupted actual speech inference.

Using a separate public development meeting, we generated donor artifacts with a different model and a different inference setting, then ran two integrated ASR interruption-and-recovery jobs. The first completed inference for one chunk, received SIGKILL, had an artifact replaced with one from another model, and recovered through combined validation. The second completed inference for three chunks, received SIGKILL, had an artifact replaced with one from another setting, and recovered through full restart. Both recovered outputs matched the reference transcript and structured output. These development cases demonstrate end-to-end interruption and recovery under different conditions.

The two ASR development jobs used one meeting but differed in fault type, interruption position, and policy. Table C2 therefore demonstrates successful integration, not a matched estimate of one policy's advantage. Both jobs recomputed 120 seconds of audio. The 76 service-state samples support the reported operational checks; they are not independent ASR samples or measurements of service request latency.

VI Discussion and Threats to Validity

1 Interpretation of the Research Questions

For RQ1, E1 shows that normal termination can coexist with an incorrect committed transcript when reuse checks only syntax. For RQ2, the valid chunks remaining after interruption explain the reduction in recomputation and the corresponding latency changes. Combined validation preserved all tested reference transcripts, but E1 does not show that P caused this preservation or that combined validation outperforms a correctly configured task-aware workflow.

The syntax comparator checks only parseability and the types of text and segments. More extensive cross-field consistency checks may detect faults that this minimal comparator misses. Accordingly, the observed contrast should be interpreted against this exact implementation, rather than generalized to all syntax validators. A fair incremental comparison requires the output-hash-only policy in E2 and equivalent task information in any workflow-tool comparator.

2 Scope of Metrics and Reclassification of Earlier Results

Transcript preservation is measured against the same model's uninterrupted output and may therefore retain its original recognition errors. Word error rate (WER) and character error rate (CER), by contrast, measure recognition quality against a reference transcript. Diarization error rate (DER) additionally requires speaker-time references, defined evaluation intervals, and an explicit policy for overlapping speech. The Korean recognition, diarization, and summarization scores reported in the earlier manuscript have been classified as requiring revalidation of their scoring procedures.

The earlier PoC reported automatic speaker-count agreement of 0/8 and cross-chunk

speaker-linking DER of 74.05%. We retain these diagnostic results under their original conditions and distinguish them from E1's recovery evaluation. Speaker-linking performance requires further validation before the system can be assessed for identifying individuals or automatically assigning responsibilities.

3 Generalization and Statistical Interpretation

Generalization is limited by the 16 recordings, their four related series, the first 120-second intervals, and independent decoding of 30-second chunks. We use descriptive recording-level summaries rather than significance tests that treat all 912 executions as independent. Longer, independently selected material is needed to assess contextual dependencies and sustained operation; Korean construction data are needed for domain-effectiveness claims.

Prior inspection of the test data and intermediate E1 results also influenced the supplementary design. Validation on an independent corpus should therefore fix the model and experimental conditions before evaluation and disclose any prior access to the data or reference annotations.

The balanced artificial fault matrix estimates behavior under specified interventions, not real-world failure rates or average operational savings. E2 reuses the same 16 recordings, so completing it can isolate validation effects but cannot by itself establish corpus-level or domain generalization.

4 Operational and Security Limitations

Absolute latency on a shared, CPU-limited host depends on hardware, load, cache state, and execution order. Service monitoring compared current execution states with previously recorded states as an operational safeguard. Effects on service request latency and availability require separate measurement.

E1 and E3a examine process termination while assuming a trusted controller and trusted validation records. They do not establish protection against simultaneous forgery of artifacts and records, host compromise, or data loss caused by power failure or storage-device damage. Deployment additionally requires procedures for distributing inputs and models, separating privileges, retaining logs, managing keys, and responding to incidents.

5 Reproducibility and Research Ethics

The companion E1 reproducibility bundle contains the frozen protocol, input and schedule manifests, original worker source, all 912 result records, descriptive aggregates, and a local analysis verifier. It also identifies the 16 reference runs and links recorded package versions to the frozen image. Audio, model weights, transcript text, credentials, and internal connection information are excluded. The included manifests identify inputs without redistributing them. A bundle inventory records SHA-256 hashes and the provenance of sanitized extracts.

This bundle supports inspection and reanalysis of the reported evidence; it is not a complete portable ASR environment or a claim of independent reproduction. The exact Python patch version and a full dependency/build lock for the historical image remain unresolved in the retained metadata examined here. External archival access, redistribution rights for code, and an independent rerun must be resolved before describing the implementation as publicly reproducible. The application architecture derives from the earlier conference PoC [17].

6 Total Cost and Conditions for Comparing Existing Methods

An operational cost comparison must account consistently for normal execution, pre-interruption inference, and post-fault inference, model loading, validation, storage, and control. This paper uses E1 post-fault latency as its primary timing measure and reports a post hoc analysis of total elapsed time, including pre-interruption cost, in Appendix E. Estimating average operational savings also requires measurements of fault frequency.

A future comparison with a workflow system must expose the same inputs, model, settings, worker, chunk dependencies, and resource limits to each method. It should distinguish task-key invalidation, output-content checking, and restoration from an intact cache rather than crediting or penalizing a tool for a feature not configured. The companion comparison specification defines this design and its freeze criteria. No comparative superiority is inferred here from documentation alone.

VII Conclusion

This study characterized transcript-artifact recovery within an on-premises meeting intelligence PoC. Using the same ASR worker and a specified fault matrix, it separated process completion from exact transcript preservation and measured recomputation and paired latency. All 912 executions terminated; 96 syntax-based runs nevertheless committed incorrect transcripts after content modification. Combined validation preserved transcripts across the tested matrix and reduced recomputation when valid chunks remained. The results provide a traceable empirical account of these behaviors rather than a new hashing or checkpointing principle.

The evidence supports content validation and selective reuse under the stated trusted-controller, independent-chunk conditions. It does not isolate the benefit of execution-condition checks, establish superiority over task-aware workflow systems, or validate whole-system security and Korean construction-domain effectiveness. The separately frozen E2 comparison and future equivalent-workflow and independent-data studies address these remaining questions.

Generative AI Assistance

OpenAI ChatGPT/Codex assisted with manuscript drafting, translation, language revision, development of experimental and analysis code, and discussion of study design and interpretation. These uses extended beyond grammar correction. Tool-generated suggestions and code are not treated as scientific evidence; the reported findings are based on the retained execution records. The companion disclosure record identifies the documented assistance and the model/version and author-verification details that require confirmation before submission.

References

[1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, Robust Speech Recognition via Large-Scale Weak Supervision, Proceedings of the 40th International Conference on Machine Learning, PMLR, Vol. 202, pp. 28492–28518, 2023. https://proceedings.mlr.press/v202/radford23a.html

[2] M. Bain, J. Huh, T. Han, and A. Zisserman, WhisperX: Time-Accurate Speech Transcription of Long-Form Audio, INTERSPEECH 2023, pp. 4489-4493, 2023. https://doi.org/10.21437/Interspeech.2023-78

[3] H. Bredin, pyannote.audio 2.1 Speaker Diarization Pipeline: Principle, Benchmark, and Recipe, INTERSPEECH 2023, 2023. https://doi.org/10.21437/Interspeech.2023-105

[4] E. N. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson, A Survey of Rollback-Recovery Protocols in Message-Passing Systems, ACM Computing Surveys, Vol. 34, No. 3, pp. 375-408, 2002. https://doi.org/10.1145/568522.568525

[5] M.-C. Hsueh, T. K. Tsai, and R. K. Iyer, Fault Injection Techniques and Tools, Computer, Vol. 30, No. 4, pp. 75-82, 1997. https://doi.org/10.1109/2.585157

[6] Bazel Contributors, Remote Caching, Bazel Documentation. https://bazel.build/remote/caching (accessed 2026-10-29)

[7] P. Di Tommaso, M. Chatzou, E. W. Floden, P. P. Barja, E. Palumbo, and C. Notredame, Nextflow Enables Reproducible Computational Workflows, Nature Biotechnology, Vol. 35, pp. 316-319, 2017. https://doi.org/10.1038/nbt.3820

[8] Nextflow Contributors, Caching and Resuming, Seqera Documentation. https://docs.seqera.io/nextflow/cache-and-resume (accessed 2026-10-29)

[9] Iterative, Running Pipelines, DVC Documentation. https://doc.dvc.org/user-guide/pipelines/running-pipelines (accessed 2026-10-29)

[10] Seqera, Getting Started with Data Lineage, Nextflow Documentation, experimental feature

introduced in version 25.04. https://docs.seqera.io/nextflow/tutorials/data-lineage (accessed 2026-10-29)

[11] Snakemake Contributors, Between Workflow Caching, Snakemake 8.20.4 Documentation. https://snakemake.readthedocs.io/en/v8.20.4/executing/caching.html (accessed 2026-10-29)

[12] T. S. Pillai, V. Chidambaram, R. Alagappan, S. Al-Kiswany, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications, OSDI 14, pp. 433-448, 2014. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/pillai

[13] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, Zero Trust Architecture, NIST SP 800-207, 2020. https://doi.org/10.6028/NIST.SP.800-207

[14] AMI Consortium, AMI Corpus. https://groups.inf.ed.ac.uk/ami/corpus/ (accessed 2026-10-29)

[15] pyannote Contributors, AMI Diarization Setup, a fork of the BUT Speech@FIT setup, revision 67c2d539286e89f68952d5dcf83912bd9f01dfae. https://github.com/pyannote/AMI-diarization-setup/tree/67c2d539286e89f68952d5dcf83912bd9f01dfae

[16] S. H. Hurlbert, Pseudoreplication and the Design of Ecological Field Experiments, Ecological Monographs, Vol. 54, No. 2, pp. 187-211, 1984. https://doi.org/10.2307/1942661

[17] J. Kim, J. Yun, J. Lee, and K. Kim, A Secure On-Premises Meeting Intelligence Pipeline Using Edge AI for Construction Enterprises, Proceedings of the 74th Summer Conference of the Korea Society of Computer and Information, Jeju, Republic of Korea, July 2026.

Appendix A Evaluation Scope and Further Validation

Tables 3 and 4 derive from the completed E1 experiment; Table C2 reports two completed integration jobs using a separate public development meeting. Each protocol is summarized separately using its own denominator. The evidence reported here is limited to results completed and reviewed when the manuscript was finalized on October 29, 2026.

E2 remains separate from the completed E1 and development evidence. Its original schedule, incorrect outcomes, execution failures, and incomplete work are retained without replacing failed attempts with successful reruns. Later operational events and partial E2 results are not imported into this manuscript's October 29 evidence cutoff.

Quality assessment against official reference annotations is a separate task. Equivalence between scores obtained with the existing custom normalization and the original distribution's scoring procedure has yet to be established. Speaker-attribution evaluation also requires speaker-time references. The present paper evaluates transcript preservation relative to an uninterrupted run with the same input and model.

The order of aggregation matters. Consider one hypothetical recording with three paired times: restart [10,20,30] seconds and combined [11,29,22] seconds. The median times are 20 and 22 seconds, whose difference is +2 seconds. The paired differences are [1,9,-8] seconds, whose median is +1 second. E1's paired estimator averages the latter type of recording-level contrast across recordings; Table 5's absolute-time columns average the former type of median. These numbers illustrate the calculation and are not additional measurements.

Appendix B Variation Across Recordings in the Existing Results

Table B1 presents a post hoc descriptive analysis of recording-level paired differences using the estimator and eligibility criteria in Table 4. Ranges indicate observed minima and maxima. Each series mean gives equal weight to its four recordings.

Table B1. Distribution of latency differences for combined validation and means by series

Fault	Min-max (s)	EN2002	ES2004	IS1009	TS3003
Stop only, pos. 1	-29.131 ~ -8.801	-11.757	-16.722	-14.513	-21.251
Stop only, pos. 3	-73.841 ~ -39.440	-49.238	-56.353	-57.140	-66.518
Truncated JSON, pos. 1	-1.287 ~ 0.613	0.037	0.416	0.030	-0.487
Truncated JSON, pos. 3	-49.834 ~ -28.943	-37.416	-40.007	-42.350	-45.693
Text edit, pos. 1	-0.798 ~ 1.117	0.158	-0.465	0.394	0.007
Text edit, pos. 3	-51.253 ~ -28.901	-37.542	-39.787	-42.928	-45.498

Conditions with both small positive and small negative differences indicate variation around zero relative to full restart. The three technical replicates characterize variation under the specified execution conditions.

Appendix C Execution Specification and Storage-Boundary Observations

The E1 execution manifest pins the container image, model, protocol, 64 audio chunks, 912-job schedule, and worker source. A preserved engineering-pilot log from the same image digest reports PyTorch 2.12.0+cu130 and OpenAI Whisper 20250625; the pilot and E1 freeze records and launch specifications share that image digest. These are image-linked historical observations, not fresh package measurements for each E1 run. The PyTorch package has a CUDA build suffix, but the experiment disables GPU access and asserts CUDA is unavailable. The exact Python patch version and a complete dependency lock have not been established from the retained records examined here. Appendix G and the bundle distinguish verified configuration from these remaining portability gaps.

E3a examines eight storage positions from each of two initial states: an empty state and a state containing a valid artifact-record pair generated under different input conditions. The file-writing process is terminated at the first seven positions; the final position is an uninterrupted control. This yields 16 cases. Temporary files are ineligible for reuse, and Table C1 describes artifacts and validation records at their final paths. New files contain synthetic data labeled with the current execution conditions; old files contain synthetic data associated with a different input.

Table C1. Expected and observed states at storage boundaries. Entries show artifact / validation-record states.

Case IDs	Interruption / control point	Empty start	Old pair
00 / 08	Before opening the artifact temporary file	None / none	Old / old
01 / 09	After partially writing the artifact temporary file	None / none	Old / old
02 / 10	After syncing the artifact temporary file, before replacement	None / none	Old / old
03 / 11	After artifact replacement, before writing its record	New / none	New / old
04 / 12	After partially writing the validation-record temporary file	New / none	New / old
05 / 13	After syncing the record temporary file, before replacement	New / none	New / old
06 / 14	After record replacement, before the completion marker	New / new	New / new
07 / 15	Uninterrupted control	New / new	New / new

All 16 cases matched the storage states in Table C1. In cases 08, 09, and 10, an old artifact remained with its matching validation record. S and H therefore accepted the pair, while P rejected its incompatible input conditions. This is direct

evidence for the specified selector logic on synthetic files after actual process termination; it is not an ASR benchmark or evidence of durability after host power loss. Case IDs e3a-plan-00 through e3a-plan-15 identify the retained records.

Both development ASR integration jobs used the same public development meeting. In the different-model case, tiny.en generated the donor artifact and large-v3-turbo was the recovery target. In the different-setting case, an artifact generated by large-v3-turbo with beam size 2 was substituted into a task requiring beam size 1. Interruption followed inference for the first and third chunks, respectively. The corresponding recovery policies were combined validation and full restart. Table C2 reports the observed interruption-and-recovery integration and execution costs for these two development cases.

Table C2. Cost breakdown of the development ASR integration jobs

Development case	Pre-stop inference (s)	Recovery inference (s)	Total inference (s)	Host (s)
Different model; pos. 1; combined	25.811	116.741	142.552	155.433
Different setting; pos. 3; full restart	85.552	117.222	202.775	215.426

Appendix D Relationship Between the PoC Materials and This Experiment

Fig. 6 shows the PoC equipment used to illustrate the enterprise meeting-transcription context: a Raspberry Pi 5 board and a development setup connecting a GIGABYTE DGX Spark to an audio interface and capture devices.

The project interim report identifies speech capture in secure environments, transcription, speaker differentiation, and meeting summarization as development objectives. It documents checks of a meeting-record list and interfaces displaying participant-specific transcripts and summaries. These materials illustrate how transcript artifacts are viewed and summarized downstream. Because reuse of an incorrect intermediate artifact can propagate errors to later stages, the present evaluation focuses on transcript-file reuse decisions and recovery cost.

The quantitative evaluation uses public AMI recordings. The PoC photographs document the application environment linking speech-capture devices with computing equipment.



(a) (b)

Fig. 6. PoC equipment: (a) Raspberry Pi 5 board; (b) GIGABYTE DGX Spark and connected peripherals.

Source: PoC equipment photographs supplied by the authors and the Collaboration Project Interim Report (August 17, 2026).

Appendix E Post Hoc Analysis of Total Elapsed Time

This post hoc analysis reexamined the 912 archived result.json files after E1 was completed. Each condition contains 48 runs, with three replicates for each of 16 recordings. For runs returning correct results, we calculate the replicate median for each recording and then average these medians with equal weights across all 16 recordings. Values are in seconds, with policies ordered as full restart, syntax-based reuse, and combined validation. Values outside parentheses denote post-fault latency L; those in parentheses denote

total elapsed time T, including pre-interruption inference.

Stop only, position 1: 83.63 (104.01), 67.39 (87.73), 67.64 (87.97).

Stop only, position 3: 83.52 (145.20), 26.19 (87.80), 26.19 (87.93).

JSON truncation, position 1: 83.55 (103.82), 83.59 (103.90), 83.49 (103.80).

JSON truncation, position 3: 83.49 (145.36), 42.06 (103.68), 42.12 (103.95).

Content modification, position 1: 83.48 (103.77), N/A, 83.49 (103.81).

Content modification, position 3: 83.53 (145.24), N/A, 42.09 (103.76).

Under content modification, all 48 syntax-based runs at each position returned incorrect transcripts, so timing conditional on correct recovery is marked N/A. These failures remain included in the overall outcomes and cost totals. The cumulative elapsed time of the 96 runs was 8417.51 seconds.

Summing per-run T across all 48 no-fault controls and 864 fault-injection runs gives 95043.46 seconds (26.40 hours). This cumulative duration includes both pre-interruption inference and recovery. SHA-256 identifiers for the raw records, condition-level and recording-level aggregates, and analysis code are retained with the separate reproducibility materials.

All E1 policies write validation records to provide consistent instrumentation. Isolating the cost of record writing and the full validation mechanism requires a no-fault comparison that varies only validation functionality while retaining the same storage procedure.

Appendix F Transcription and Summarization Service Interface

Fig. 7 shows the development interface using lecture audio. The central transcript panel is labeled 스크립트 (Transcript), and the right panel is labeled AI 요약 (AI Summary). Other visible labels are 전체 기록 (All Records), 처리 중 (Processing), 완료된 기록 (Completed Records), 새 녹음 (New Recording), 전체 핵심 요약 (Overall Summary), and 주요 내용 (Key Points). The image illustrates downstream use of stored transcripts; it does not evaluate Korean construction-meeting recognition or summary quality.

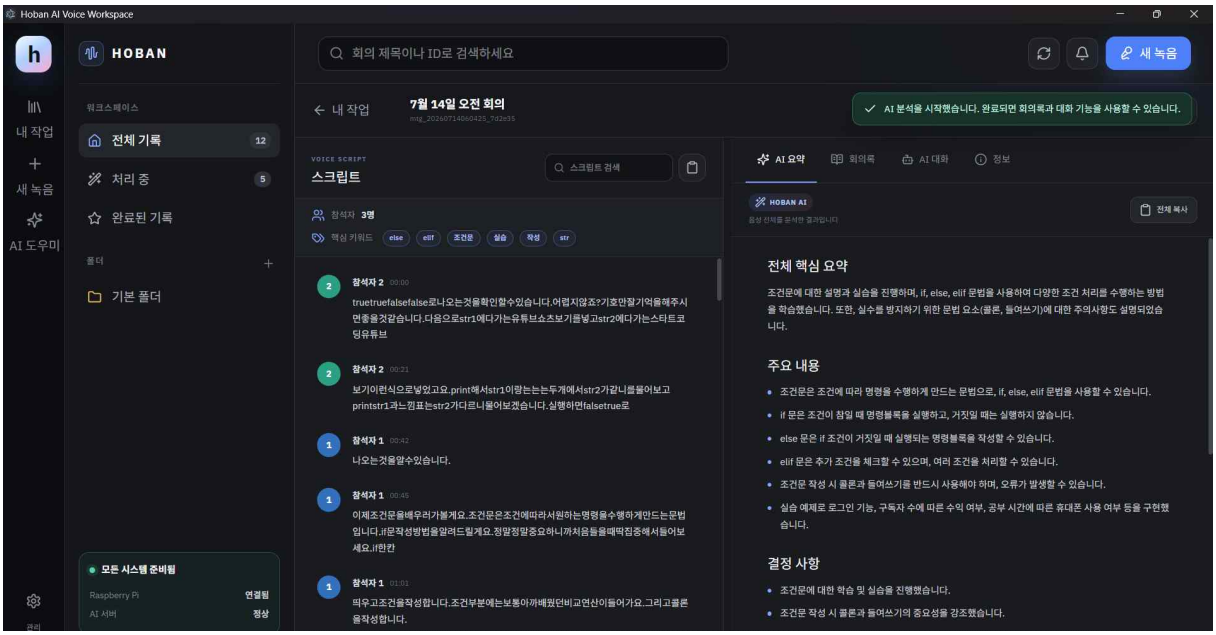


Fig. 7. PoC development interface: record navigation (left), chronological transcript (center), and generated summary (right). The paragraph above translates the original Korean labels.

Source: Service screenshot supplied by the authors (development material from July 2026).

Appendix G Reproduction Guide and Evidence Map

G1. Reuse procedure (E1). For each chunk: (1) construct expected binding B from the frozen manifest and protocol; (2) initialize reuse = false; (3) if the policy permits reuse and the artifact exists, parse it and check that it is an object with string text and list segments; (4) for combined validation, also read the ledger and require ledger.binding = B, artifact.binding = B, and ledger.output_sha256 = SHA-256(artifact bytes); (5) on a failed check or the handled parsing/file/key/type errors, infer that chunk again; otherwise reuse it; (6) save each recomputed artifact, then its ledger, and continue. Full restart never reuses; syntax-based reuse stops at the format checks. Empty text is allowed for silence. The archived code is authoritative for the exact exception handling and write order.

G2. Stored schema. An artifact has {"binding": B, "text": string, "segments": list}; its ledger has {"binding": B, "output_sha256": SHA-256 of the complete artifact file}. For the first chunk of EN2002a, B.id is EN2002a_00; B.audio_sha256 begins e6884e5d, B.model_sha256 begins aff26ae4, and B.protocol_sha256 begins 818aa7ee. These prefixes are for readability only. The companion schema example and input manifest contain all 64

hexadecimal characters. Validation hashes the original serialized file bytes rather than a reserialized JSON object.

G3. Inputs and references. The input manifest specifies 64 WAV chunks, including parent and chunk hashes, start time, duration, and sample rate. Each of EN2002a-d, ES2004a-d, IS1009a-d, and TS3003a-d contributes four contiguous 30-second chunks beginning at 0, 30, 60, and 90 seconds. Reference IDs have the form <recording>__none__full_restart. The 16 references are part of the 48 no-fault controls, followed by 864 scheduled fault runs. Reference equality means equality of the ordered four-string list, not word error rate against a human transcript.

G4. Frozen execution configuration. E1 uses large-v3-turbo, CPU inference, two intra-operation threads and one inter-operation thread, English transcription, temperature 0, beam size 1, fp16 disabled, and condition_on_previous_text disabled. Each execution has a 600-second limit. The container uses runc, no network, no GPU visibility, a 2-CPU quota, and 8-GiB memory and memory-plus-swap limits. These are the E1 settings; later studies have separately frozen execution limits. Full image, model, protocol, source, and schedule hashes are in the bundle.

G5. Environment evidence. PyTorch 2.12.0+cu130 and Whisper 20250625 are reported by a retained pilot executed from the image also pinned for E1. The bundle records this chain of evidence, rather than treating package versions on the current host as historical E1 versions. The exact Python patch version and a full dependency/build lock remain unverified. The bundle reproduces the analysis from retained records without loading a model or rerunning ASR.

G6. E1 evidence. Purpose: characterize transcript preservation and recovery cost. Inputs: 16 recordings from four series, totaling 32 minutes of unique audio. Denominator: 48 no-fault controls plus 864 fault runs. Comparators: full restart, syntax-based reuse, and combined validation with the same worker. Supported claim: behavior and cost under the specified interventions. Unsupported extension: a causal estimate of P alone, a population failure rate, or superiority over existing workflow systems.

G7. E3a evidence. Purpose: inspect the states of an artifact and its record at eight storage positions from each of two initial states. Denominator: 14 actual process terminations and 2 uninterrupted controls using synthetic files; no speech recordings. Supported claim: observed file states and selector decisions at the tested boundaries. It supplies neither independent ASR samples nor host power-loss evidence.

G8. ASR development evidence. Purpose: confirm that donor substitution, interruption, and recovery integrate with actual inference. Inputs: one public development meeting used in two jobs. The jobs differ in fault type, interruption position, and recovery policy. Both preserve the target transcript, but their cost difference is not a matched policy effect. These are development observations, separate from E1 and E2 main denominators.

G9. Pending E2 evidence. Purpose: compare output-hash-only with combined validation under intact but incompatible artifact-record pairs. Planned denominator: 960 main executions and 32 separate donor-generation executions using the same 16 recordings. The protocol was designed after intermediate E1 inspection. The study was incomplete at the manuscript cutoff; no E2 main result is used to establish a contribution here. More executions of these inputs do not provide independent-corpus generalization.

G10. Reanalysis and access. The companion E1_Reproducibility_Bundle.zip supplies the frozen worker and manifests, 912 original result records, aggregates, and a standard-library analysis verifier. It excludes audio, weights, and transcript content. Public archival access and a portable inference environment remain pre-submission tasks.

Authors



Jaehwan Kim is a Ph.D. candidate at aSSIST (Seoul School of Integrated Sciences and Technologies), Seoul, Republic of Korea. He contributed to the overall system design, pipeline implementation, empirical measurements, and manuscript preparation. His research interests include on-premises meeting intelligence and edge AI.



Jaeseong Yun is an AI engineer at XAIKOREA Inc., Seongnam, Republic of Korea. He contributed to speech AI processing, vLLM serving, and concurrent processing for meeting summarization.



Jongseok Lee leads the Infrastructure Team at RAD CNS Co., Ltd., Republic of Korea. He contributed to the two-node GPU infrastructure and service orchestration.



Kyungmin Kim is a principal researcher in the Technical Support Team at Security Platform, Republic of Korea. He contributed to the security architecture, AES-256-GCM protection, and Vault KEK integration.